

N7PD : Miniprojet SAT

Nicolas Amat & Kevin Delmas

1 Utilisation de Z3 comme un solver SAT

Nous allons dans la suite utiliser le solver SMT Z3, un démonstrateur de théorèmes développé par Microsoft Research. Nous allons dans un premier temps l'utiliser comme un solver SAT, i.e. sans théorie du premier ordre. Nous utiliserons l'API Python de Z3 dont la documentation est disponible à l'URL <https://z3prover.github.io/api/html/namespacez3py.html>. Toutes les classes de l'API Python de Z3 appartiennent au paquetage `z3`.

1.1 Configuration de votre environnement de développement

Un exemple d'utilisation de l'API est fourni dans le fichier `try-z3.py` que vous avez à disposition. Vous pouvez l'exécuter dans un environnement virtuel comme suit :

```
1 python3 -m venv myEnv
2 source myEnv/bin/activate
3 pip3 install z3-solver
4 python3 try-z3.py
```

1.2 Créer des variables booléennes

Pour pouvoir interagir avec le solver, il faut importer l'ensemble des éléments fournis par l'API Python

```
1 from z3 import *
```

manipuler un objet de type `Solver` représentant une instance du solver.

```
1 # define problem
2 s = Solver()
```

On utilise le contexte pour pouvoir créer des *expressions booléennes* qui seront ici de simples variables booléennes:

```
1 # define variables
2 a = Bool('a')
3 b = Bool('b')
4 c = Bool('c')
```

1.3 Créer des formules

Pour créer des formules, on utilise encore une fois le contexte. Par exemple, si l'on veut créer $a \rightarrow b$ et $\neg b \vee \neg c$:

```
1 s.add(Implies(a, b))
2 s.add(Or(Not(b), Not(c)))
3 s.add(a)
```

On peut remarquer que `Or`, comme `And`, a un paramètre variadique, donc on peut lui passer autant d'arguments que l'on souhaite ou un tableau contenant des instances de `BoolExpr`.

1.4 Vérifier si un ensemble de formules est satisfaisable

Ensuite, il suffit d'appeler la méthode `check` sur le solver et de vérifier le résultat (ces instructions sont encapsulées dans la méthode `checkAndPrint` dans l'exemple fourni):

```
1 # solving
2 checkAndPrint(s)
```

Ici, l'ensemble $\{a \rightarrow b, \neg b \vee \neg c, a\}$ est satisfaisable. Un modèle est affiché au format SMT-LIB [?]. De la même façon, si l'ensemble n'était pas satisfaisable, une preuve d'incohérence serait affichée au même format.

1.5 Sauvegarder l'état du solver

On peut stocker l'état du solver à travers un mécanisme de pile. Par exemple, pour sauvegarder l'état courant du solver. On peut ensuite ajouter une nouvelle formule (c ici), vérifier la satisfiabilité du problème:

```
1 # making the problem UNSAT
2 print("adding c...")
3 s.push
4 s.add(c)
5 checkAndPrint(s)
```

Il est possible de revenir à l'état précédent en retirant toutes les contraintes ajoutées depuis le dernier `push`:

```
1 s.pop
2 checkAndPrint(s)
```

1.6 Récupérer un modèle propositionnel

Si le problème est SAT c'est-à-dire que `s.check == sat`, on peut récupérer via `s.model` un modèle propositionnel satisfaisant l'ensemble des formules dans le solver. Cette interprétation des variables booléennes est un dictionnaire associant à chaque variable une valeur. Attention, une variable comme `a` n'a elle-même pas de valeur booléenne.

```
1 def checkAndPrint(s):
2     # solving
3     result = s.check()
4
5     if result == sat:
6         print("SAT!")
7         # get the model
8         m = s.model()
9         print('a={0}, b={1}, c={2}\n'.format(m[a],
10                                              m[b],
11                                              m[c]))
12     else:
13         print("UNSAT!\n")
```

2 Résolution de grilles de Sudoku

Nous allons utiliser Z3 pour résoudre un problème classique : compléter une grille de Sudoku. Un exemple de grille pouvant être utilisée est donné sur la figure 1.

						8	5	1	
			1		2		9	4	
					8				
				3		6	4		
5			6				7	3	
6									
	4	3	7	2					
	2	9	8			3			

Figure 1: Grille Sudoku du journal Le Monde du 8 novembre 2016 (grille élaborée par Yan Georget)

Une grille de Sudoku construite à partir d'un $n > 0$ donné à les propriétés suivantes :

- la grille est de taille $n^2 \times n^2$
- les valeurs pouvant être placées dans la grille appartiennent à $\{1, \dots, n^2\}$
- chaque case doit avoir une valeur
- chaque valeur doit apparaître une et une seule fois dans une ligne
- chaque valeur doit apparaître une et une seule fois dans une colonne

- chaque valeur doit apparaître une et une seule fois dans les sous-grilles de taille $n \times n$ (on parle de sous-grilles ne se recouvrant pas)

Vous trouverez dans l'archive fournie le script `sudoku-z3.py`. En particulier, vous disposez des méthodes suivantes:

- une méthode `pretty_print` permettant d'afficher le modèle du solver s'il existe (affichage de la grille solution). S'il n'y a pas de valeur trouvée pour une case, le symbole `_` s'affiche. S'il y a plusieurs valeurs pour une case, la première valeur s'affichera en vidéo inverse, par exemple **1**.
- une méthode `read_sudoku` permettant de charger un fichier au format CSV représentant une grille

Des commentaires `# FIXME` sont situés aux endroits que vous devez compléter.

Question 1: Lire le code et comprendre les méthodes existantes.

Question 2: Lancer le code dans votre environnement virtuel :

```
1 python3 sudoku-z3.py
```

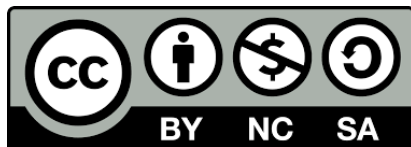
Que pensez-vous de la solution fournie?

Question 3: ajouter les formules logiques permettant de spécifier les règles du jeu.

Question 4: vérifier sur la grille représentée sur la figure 1 que le solver trouve une solution.

Question 5: regarder le temps nécessaire pour créer les contraintes et le temps nécessaires pour résoudre le problème. Que constatez-vous?

Question 6: le fichier exemple `multiple-sol.csv` contient une grille ayant plusieurs solutions (ce n'est donc pas une vraie grille de Sudoku). Comment faire pour toutes les énumérer?



This work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported license (CC BY-NC-SA 3.0)

You are free to Share (copy, distribute and transmit) and to Remix (adapt) this work under the following conditions:

- Attribution– You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).
- Noncommercial– You may not use this work for commercial purposes.
- Share Alike– If you alter, transform, or build upon this work, you may distribute the resulting work only under the same or similar license to this one.

See <https://creativecommons.org/licenses/by-nc-sa/3.0/>.